LATTICE: Ledgered Agent Traces for Transparent, Inspectable Collaborative Execution

Content-Addressed Accountability for Multi-Agent AI Systems

Ali Murtaza Bhutto
alimurtazabhutto@proton.me
ORCID: 0009-0007-2787-943X

Independent Researcher

April 2026

Abstract

Multi-agent AI systems increasingly make consequential decisions, yet lack mechanisms to explain *why* a particular conclusion was reached, what evidence supported it, or whether that evidence has since been invalidated. We present LATTICE, an open-source accountability layer that sits beneath any agent framework and records every agent decision as a content-addressed, cryptographically signed node in a directed acyclic graph (DAG). Each node, called a *Claim*, carries a SHA-256 content hash, an Ed25519 digital signature, a calibrated confidence score, provenance metadata, and references to supporting evidence or upstream claims. LATTICE provides four core capabilities: (1) backward traceability from any conclusion to raw evidence, (2) cryptographic tamper detection through content addressing and signature verification, (3) a revocation waterfall that propagates compromise status through the dependency graph when upstream evidence is invalidated, and (4) zero-friction runtime monitoring that instruments arbitrary Python functions without altering their behavior. We describe the design, architecture, and implementation of LATTICE, evaluate its performance characteristics, and discuss its relationship to formal verification of autonomous systems. The system is framework-agnostic, requires no external services, and is released under the MIT license.

Keywords: multi-agent systems, provenance, accountability, content addressing, cryptographic signatures, directed acyclic graphs, runtime monitoring, formal verification

1 Introduction

The rapid deployment of multi-agent AI systems in domains ranging from open-source intelligence (OSINT) to cybersecurity operations to automated research has created an accountability gap. These systems combine large language model (LLM) reasoning, autonomous planning, tool invocation, memory, and strategic decision-making to produce conclusions that inform real-world actions. Yet the reasoning chains that produce those conclusions are typically opaque: there is no standard mechanism to determine what evidence an agent used, how confident it was, whether its reasoning was sound, or whether the evidence it relied on remains valid.

This accountability gap is not merely an engineering inconvenience. In regulated industries, investigative journalism, intelligence analysis, and security research, an unverifiable claim is worse than no claim at all. When an automated system asserts that a domain is malicious, that a social media account is part of a coordinated campaign, or that a vulnerability exists in critical infrastructure, the consumers of that assertion need to trace it backward to its evidential foundations, verify that no step in the reasoning chain has been tampered with, and determine whether any supporting evidence has since been retracted or invalidated.

Existing approaches to this problem fall into three categories, none of which is fully satisfactory:

1. **Logging.** Systems that record agent actions in flat logs or structured event streams. These provide a record of what happened but lack the graph structure needed to trace dependencies,

the cryptographic guarantees needed to detect tampering, and the uncertainty quantification needed to assess reliability.

2. **Provenance standards.** The W3C PROV data model [1] provides a vocabulary for describing provenance relationships, but it does not prescribe content addressing, cryptographic integrity, or confidence calibration. Adapting PROV to multi-agent AI systems requires substantial additional infrastructure.

3. **Formal verification.** Model checking [2] and runtime verification [3] provide rigorous guarantees about system behavior, but current techniques were developed for systems with well-defined state spaces and deterministic transitions. Adapting them to hybrid symbolic-subsymbolic architectures with stochastic behavior remains an open research problem [4].

LATTICE addresses this gap by providing a lightweight accountability layer that can be inserted beneath any agent framework. It records every agent decision as a *Claim*: a content-addressed, cryptographically signed assertion that references its supporting evidence through a directed acyclic graph. The design philosophy is that accountability should be a property of the infrastructure, not a burden on the agent developer.

The contributions of this paper are:

1. A data model for accountable multi-agent execution based on content-addressed, cryptographically signed claim DAGs with calibrated confidence scores (Section 3).

2. An architecture and implementation that provides backward traceability, tamper detection, revocation propagation, and zero-friction runtime monitoring with minimal performance overhead (Sections 4 and 5).

3. An evaluation of the system's performance characteristics and a discussion of its relationship to formal verification (Sections 6 and 7).

The system is released as open-source software under the MIT license at https://github.com/thunderstornX/lattice.

2 Related Work

2.1 Provenance and Lineage

Data provenance has a long history in database systems [5] and scientific workflows [6]. The W3C PROV standard [1] provides a general-purpose data model for provenance that distinguishes entities, activities, and agents. Several systems implement PROV for specific domains, including ProvDB for database queries [7] and PROV-IO for I/O operations.

LATTICE differs from PROV-based systems in three respects: (1) it uses content addressing to make provenance relationships tamper-evident, (2) it integrates cryptographic signatures to provide agent attribution, and (3) it includes confidence calibration as a first-class property of every claim. These additions are motivated by the adversarial threat model inherent in multi-agent AI systems, where agents may produce hallucinated content, and by the uncertainty inherent in LLM-based reasoning.

2.2 Content-Addressed Storage

Content addressing, in which data is identified by its cryptographic hash rather than its location, is a foundational concept in distributed systems. Git [8] uses SHA-1 hashes to identify commits, trees, and blobs, creating an immutable history of changes. The InterPlanetary File System (IPFS) [9] generalizes this idea to a distributed content-addressed file system using Merkle DAGs.

LATTICE applies content addressing to the specific domain of multi-agent decision trails. Each Claim's identifier is the SHA-256 hash of its canonical content, ensuring that any modification to the claim's assertion, evidence references, confidence, or metadata produces a different identifier. This property enables efficient tamper detection: verifying a claim requires only recomputing its hash and comparing it to the stored identifier.

2.3 Structured Analytic Techniques

The intelligence community has developed structured analytic techniques (SATs) [10] to mitigate cognitive biases in human analysis. Techniques such as Analysis of Competing Hypotheses (ACH), Key Assumptions Check, and Devil's Advocacy impose structured reasoning disciplines on analysts. LATTICE's audit mechanism, which flags unsup-

ported claims, low-confidence assertions, and broken evidence references, can be seen as an automated implementation of similar principles applied to multi-agent systems.

2.4 Runtime Verification

Runtime verification (RV) [3] monitors system behavior during execution to check compliance with formal specifications. Recent work on runtime verification of multi-agent systems has addressed communication protocol compliance [11, 12] and holistic verification of autonomous cognitive systems [4]. The VITAMIN framework [13] provides compositional model checking for multi-agent systems.

LATTICE's runtime monitoring decorator (@lattice_monitor) can be understood as a lightweight form of runtime verification: it instruments agent functions to capture their inputs, outputs, and provenance relationships as verifiable claims. While LATTICE does not perform formal property checking, the claim DAG it produces provides the raw material for downstream formal analysis.

2.5 Blockchain and Distributed Ledgers

Blockchain systems provide tamper-evident ledgers through consensus mechanisms and cryptographic chaining. While LATTICE shares the goal of tamper evidence, it deliberately avoids the complexity, latency, and resource consumption of distributed consensus. LATTICE is designed for single-user, local investigations where the threat model is post-hoc verification rather than real-time Byzantine fault tolerance. The SQLite-backed store provides the performance characteristics needed for interactive analysis without the overhead of a distributed ledger.

3 Design

3.1 Design Principles

LATTICE is guided by four principles:

Accountability is infrastructure. Recording provenance, signing claims, and maintaining the dependency graph should happen automatically, without requiring agent developers to modify their code beyond adding a decorator.

Provenance is inseparable from meaning. A claim without documented evidence is an unsupported assertion. LATTICE enforces this by making evidence references a structural property of every claim, and by flagging claims with no evidence as audit issues.

Uncertainty is a first-class object. Every claim carries a confidence score in $[0.0, 1.0]$. Rather than producing binary labels ("malicious" or "benign"), agents express calibrated uncertainty that propagates through the dependency graph.

Immutability preserves integrity. Claims, once created, are never modified. Revocation status is stored in a separate table, leaving the original claim and its cryptographic signature intact. This design ensures that the historical record is always available for audit, even when specific claims have been retracted.

3.2 Core Data Model

The LATTICE data model consists of three entity types:

Evidence. A content-addressed blob of raw data (tool output, API response, document text). Each evidence blob is identified by the SHA-256 hash of its content, ensuring deduplication and tamper detection.

$$\text{evidence_id} = \text{SHA-256}(\text{data}) \tag{1}$$

Claim. The fundamental unit of accountability. A Claim is a signed assertion that references its supporting evidence:

$$\text{claim_id} = \text{SHA-256}(\text{canonical}(C)) \tag{2}$$

where canonical(C) is the deterministic JSON serialization of the claim's content fields: `agent_id`, `assertion`, `evidence` (sorted), `method`, `timestamp`, and `metadata`. The claim also carries an Ed25519 signature over the claim_id, binding the claim to its originating agent.

Formally, a Claim is a tuple:

$$C = (id, a, s, E, \gamma, m, t, \mu, \sigma)$$

where id is the content hash, a is the agent identifier, s is the assertion string, $E = \{e_1, \ldots, e_k\}$ is the set of evidence references (which may be evidence IDs or other claim IDs), $\gamma \in [0, 1]$ is the confidence score, m is the derivation method, t is the timestamp, μ is arbitrary metadata, and σ is the Ed25519 signature.

Agent. A registered entity with a unique identifier, a role label, and an Ed25519 keypair. The private key is used to sign claims; the public key is stored for verification.

3.3 The Claim DAG

Claims reference other claims and evidence blobs through their evidence lists, forming a directed acyclic graph:

- **Leaf nodes** are raw evidence blobs.
- **Intermediate nodes** are derived analyses that reference other claims or evidence.
- **Root nodes** are final conclusions.

The DAG structure enables backward traceability: given any conclusion, one can walk the graph backward through all intermediate analyses to the raw evidence that supports it. Cycle detection is enforced at insertion time to maintain the acyclic invariant.

3.4 Revocation Model

When evidence is discovered to be incorrect, fabricated, or outdated, the corresponding claim can be *revoked*. Revocation triggers a *waterfall*: all downstream claims that depend (directly or transitively) on the revoked claim are marked as *compromised*.

Formally, let $\text{deps}(c)$ denote the set of claims that transitively depend on claim c:

$$\text{deps}(c) = \{c' \mid \exists \text{ path } c' \rightsquigarrow c \text{ in the DAG}\}$$

When c is revoked, every $c' \in \text{deps}(c)$ is marked as compromised. The original claims and their signatures are never modified; revocation status is stored separately, preserving the historical record.

4 Architecture

LATTICE is organized as a layered architecture with clean separation of concerns:

Storage Layer. An SQLite database (WAL mode) with four tables: `agents`, `evidence`, `claims`, and `revocations`. SQLite provides ACID transactions, zero-configuration deployment, and sufficient performance for interactive analysis. The database is a single file, making investigations portable.

Core Layer. The `LatticeStore` class provides the primary API: agent registration, evidence storage, claim creation and retrieval, DAG traversal, audit, signature verification, and revocation. All operations are type-hinted and documented.

Instrumentation Layer. The `@lattice_monitor` decorator wraps arbitrary Python functions, automatically capturing inputs, outputs, and timing as signed claims. This enables zero-friction integration with any agent framework.

Presentation Layer. A CLI (`click` + `rich`) and a local web dashboard (FastAPI + inlined D3.js) provide human-readable access to the claim DAG. The dashboard visualizes the DAG as a force-directed graph with color-coded nodes indicating claim status.

5 Implementation

5.1 Content Addressing

Claim IDs are computed by serializing the claim's content fields to canonical JSON (sorted keys, no whitespace) and hashing with SHA-256:

```
canonical = json.dumps({
    "agent_id": ...,
    "assertion": ...,
    "evidence": sorted(evidence),
    "method": ...,
    "timestamp": ...,
    "metadata": ...,
}, sort_keys=True, separators=(",",":"))
claim_id = sha256(canonical).hexdigest()
```

Evidence IDs are simply the SHA-256 hash of the raw data string, providing automatic deduplication.

5.2 Cryptographic Signatures

Each agent is assigned an Ed25519 keypair at registration time. When a claim is created, the agent's private key signs the claim_id (which is already a hash of the claim's content):

```
signature = private_key.sign(
    claim_id.encode("utf-8")
).hex()
```

Verification recomputes the claim_id from the stored content (tamper detection) and then verifies the Ed25519 signature against the agent's public key (attribution verification).

5.3 Revocation Waterfall

The downstream propagation of revocation is implemented using a recursive common table expression (CTE) in SQLite:

```
WITH RECURSIVE downstream(claim_id) AS (
  SELECT c.claim_id
  FROM claims c, json_each(c.evidence) AS je
  WHERE je.value = :revoked_id
  UNION
  SELECT c2.claim_id
```

```
  FROM claims c2, json_each(c2.evidence) AS je2
  JOIN downstream d ON je2.value = d.claim_id
)
SELECT claim_id FROM downstream;
```

This query traverses the dependency graph from the revoked claim downward through all claims that reference it, directly or transitively. The upstream check (determining whether a given claim depends on any revoked claim) uses a similar recursive CTE walking in the opposite direction.

5.4 Cycle Detection

When a new claim is inserted, LATTICE walks the ancestor chain of each evidence reference using a breadth-first search. If the new claim's own ID appears anywhere upstream, the insertion is rejected with a `CyclicDependencyError`. This check runs in $O(|V| + |E|)$ time where $|V|$ and $|E|$ are the vertices and edges reachable from the claim's evidence references.

5.5 Runtime Monitoring

The `@lattice_monitor` decorator uses Python's `functools.wraps` and `inspect` module to transparently instrument functions:

```
@lattice_monitor(harvester, method="tool:dns")
def dns_lookup(domain: str) -> dict:
    """DNS lookup for {domain}"""
    return run_nslookup(domain)
```

The decorator: (1) executes the wrapped function normally, (2) serializes the return value as an evidence blob, (3) constructs an assertion from the function's docstring template, (4) creates a signed claim referencing the evidence, and (5) returns the original return value unchanged. The function's callers are completely unaware of the instrumentation.

5.6 Audit

The audit function scans all claims and reports three categories of issues:

- **Unsupported claims:** Claims with an empty evidence list.
- **Low confidence:** Claims below a configurable threshold (default 0.3).
- **Broken references:** Claims referencing evidence or claim IDs that do not exist in the store.

6 Evaluation

6.1 Correctness

LATTICE includes a comprehensive test suite of 78 tests covering all public APIs, including:

- Content addressing determinism and collision resistance
- Ed25519 signature creation and verification
- DAG traversal correctness (chains, diamonds, disconnected components)
- Revocation waterfall propagation (linear chains, diamond DAGs)
- Cycle detection (self-references and indirect cycles)
- CLI command output
- Dashboard API endpoint correctness
- Runtime monitor behavior (evidence capture, metadata, assertion templates)

All tests pass on Python 3.10, 3.11, 3.12, and 3.13.

6.2 Performance

Table 1 summarizes performance measurements on a standard workstation (Intel i7, 16GB RAM, SSD).

Table 1: Performance characteristics

Operation	Time
Claim creation + signing	< 1 ms
Evidence storage (dedup)	< 0.5 ms
Signature verification	< 0.5 ms
Trace (100-claim chain)	< 10 ms
Audit (1000 claims)	< 50 ms
Revocation waterfall (1000 claims)	< 100 ms
Dashboard render (100 nodes)	< 2 s

These measurements confirm that LATTICE adds negligible overhead to agent execution. The dominant cost is SQLite I/O, which is mitigated by WAL mode enabling concurrent reads.

6.3 Dependencies

LATTICE has minimal dependencies by design:

- `cryptography` – Ed25519 signatures

- `click` – CLI framework
- `rich` – terminal formatting
- `fastapi` + `uvicorn` – local dashboard
- `sqlite3` – storage (Python stdlib)

No external services, cloud dependencies, or network calls are required.

7 Discussion

7.1 Relationship to Formal Verification

LATTICE is not a formal verification tool. It does not express temporal logic specifications, perform model checking, or provide mathematical guarantees about system behavior. However, it produces the *raw material* that formal verification would operate on: a structured, content-addressed, cryptographically signed record of every agent decision, its evidence basis, and its dependency relationships.

We observe several connection points between LATTICE's accountability model and formal verification:

Claim DAGs as system traces. The LATTICE DAG is, in effect, an execution trace of the multi-agent system. Runtime verification techniques [3, 13] could be applied to these traces to check temporal properties (e.g., "every conclusion must be supported by at least two independent evidence sources").

Revocation as property violation. The revocation waterfall implements a form of backward reachability analysis. When a claim is revoked, LATTICE computes the set of all downstream claims that are now compromised. This is structurally similar to counterexample generation in model checking: given a violated property (the revoked claim), identify all conclusions that depended on it.

Confidence as uncertainty quantification. Formal verification under imperfect information [14, 15] requires explicit representation of what agents know and do not know. LATTICE's confidence scores and observability boundary documentation provide a foundation for this representation.

Audit as lightweight specification checking. The audit function checks properties that could be expressed in a formal specification language: "no claim shall have an empty evidence list" (safety property), "all evidence references shall resolve" (well-formedness), "no claim's confidence shall fall below threshold" (quality constraint).

Future work could extend LATTICE with formal specification support, enabling users to define temporal or strategic properties over the claim DAG and verify them automatically.

7.2 Limitations

Several limitations should be acknowledged:

Single-user model. LATTICE currently stores private keys in plaintext in the SQLite database. This is acceptable for local, single-user investigations but would require encryption at rest and proper key management for multi-user or networked deployments.

No confidence propagation. Confidence scores are assigned per-claim but do not automatically propagate through the dependency graph. A conclusion derived from two claims with confidence 0.7 and 0.5 still requires the agent to manually compute an appropriate confidence for the derived claim. Bayesian confidence propagation is a natural extension.

LLM-specific challenges. While LATTICE can instrument LLM calls through the `@lattice_monitor` decorator, it treats the LLM as a black box. It captures inputs and outputs but cannot verify that the LLM's internal reasoning was sound. This is a fundamental limitation shared by all monitoring approaches to LLM-based systems.

Scalability. The SQLite backend and in-memory list operations (e.g., `list_claims(limit=100000)`) impose practical limits on investigation size. For investigations with millions of claims, a more scalable storage backend would be needed.

7.3 Applications

LATTICE is designed to be domain-agnostic, but several application areas are particularly well-suited:

OSINT investigations. Open-source intelligence analysis involves collecting data from heterogeneous sources, correlating entities across platforms, and producing assessments with documented uncertainty. LATTICE provides the accountability infrastructure that current OSINT tools lack.

Security research. Vulnerability analysis, threat attribution, and incident response require

defensible conclusions backed by verifiable evidence. LATTICE's tamper-evident claim chains provide the audit trail needed for regulatory compliance and legal proceedings.

Automated research pipelines. Multi-agent systems that conduct literature reviews, data analysis, or experimental design can use LATTICE to document their reasoning chains, enabling human reviewers to verify and challenge automated conclusions.

8 Conclusion

We have presented LATTICE, an accountability layer for multi-agent AI systems that records every agent decision as a content-addressed, cryptographically signed claim in a directed acyclic graph. The system provides backward traceability, tamper detection, revocation propagation, and zero-friction runtime monitoring with minimal performance overhead. By making agent reasoning chains inspectable, traceable, and verifiable, LATTICE addresses a critical gap in the current multi-agent AI ecosystem.

The source code, documentation, and examples are available at https://github.com/thunderstornX/lattice.

Acknowledgments

The author thanks the open-source community for feedback on early versions of LATTICE, and acknowledges the influence of the W3C PROV working group, the intelligence community's structured analytic techniques, and the content-addressed storage paradigm pioneered by Git and IPFS.

References

[1] L. Moreau and P. Missier, "PROV-DM: The PROV Data Model," W3C Recommendation, April 2013. Available: https://www.w3.org/TR/prov-dm/

[2] E. M. Clarke, O. Grumberg, and D. A. Peled, *Model Checking*. MIT Press, 1999.

[3] M. Leucker and C. Schallhart, "A brief account of runtime verification," *J. Log. Algebraic Methods Program.*, vol. 78, no. 5, pp. 293–303, 2009.

[4] A. Ferrando, L. A. Dennis, R. C. Cardoso, M. Fisher, D. Ancona, and V. Mascardi, "Holistic verification and validation of autonomous systems," *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 4, pp. 1–37, 2021.

[5] P. Buneman, S. Khanna, and W. C. Tan, "Why and where: A characterization of data provenance," in *Proc. ICDT*, 2001, pp. 316–330.

[6] J. Freire, D. Koop, E. Santos, and C. T. Silva, "Provenance for computational tasks: A survey," *Computing in Science & Engineering*, vol. 10, no. 3, pp. 11–21, 2008.

[7] B. S. Arab et al., "ProvDB: Provenance-enabled lifecycle management of collaborative data analysis workflows," *IEEE Data Eng. Bull.*, vol. 41, no. 4, pp. 26–38, 2018.

[8] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014.

[9] J. Benet, "IPFS – Content Addressed, Versioned, P2P File System," arXiv:1407.3561, 2014.

[10] R. J. Heuer and R. H. Pherson, *Structured Analytic Techniques for Intelligence Analysis*. CQ Press, 2010.

[11] D. Ancona, A. Ferrando, and V. Mascardi, "Parametric runtime verification of multiagent systems," in *Proc. AAMAS*, 2017, pp. 1457–1459.

[12] A. Ferrando, D. Ancona, and V. Mascardi, "Decentralizing MAS monitoring with DecAMon," in *Proc. AAMAS*, 2017, pp. 239–248.

[13] A. Ferrando and V. Malvone, "VITAMIN: A Compositional Framework for Model Checking of Multi-Agent Systems," in *Proc. ICAART*, 2025, pp. 648–655.

[14] F. Belardinelli, A. Lomuscio, and V. Malvone, "An abstraction-based method for verifying strategic properties in multi-agent systems with imperfect information," in *Proc. AAAI*, 2019, pp. 6030–6037.

[15] F. Belardinelli and V. Malvone, "Three-valued strategic abilities," in *Proc. KR*, 2020, pp. 89–98.

[16] A. Ferrando and V. Malvone, "Runtime Verification via Rational Monitor with Imperfect Information," *ACM Trans. Softw. Eng. Methodol.*, vol. 35, no. 3, pp. 74:1–74:29, 2026.

[17] F. Mogavero, A. Murano, G. Perelli, and M. Y. Vardi, "Reasoning about strategies: On the model-checking problem," *ACM Trans. Comput. Log.*, vol. 15, no. 4, pp. 34:1–34:47, 2014.